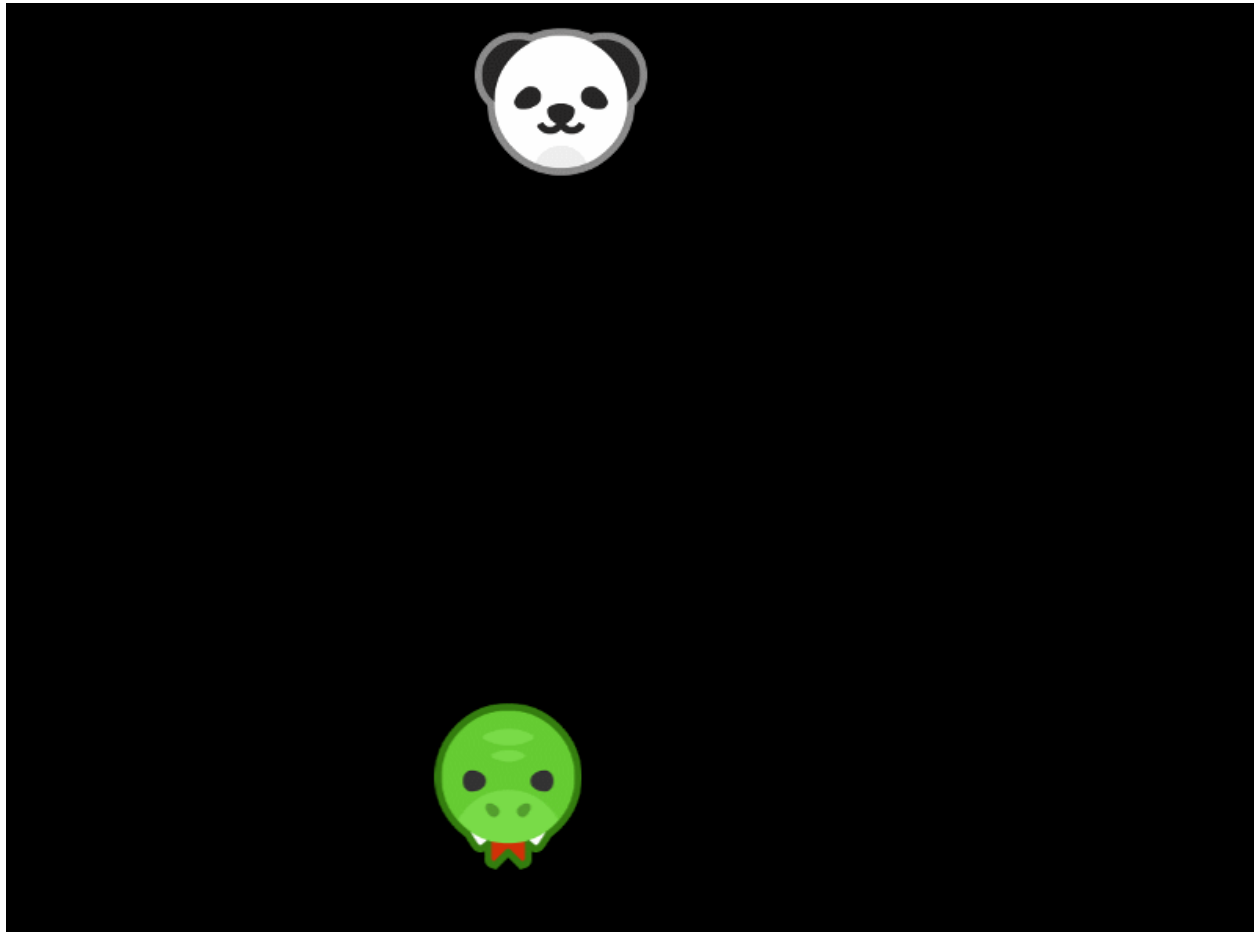


Chapter 14 - Game: Shoot the enemy

Let's use everything we learned so far to create a simple game. You can read about programming and making games all you want, but to really learn it you'll have to do it.

A game is essentially a bunch of problems that you have to solve. When you ask an experienced programmer to make PONG, he won't look up a *How to make PONG*. They can divide PONG into separate problems, and know how to solve each one of them. This chapter is to show you how to split a game into multiple tasks.

The game we'll be making is simple: An enemy is bouncing against the walls. We have to shoot it. Each time we shoot it, the enemy goes a little faster. When you miss, it's game over and you'll have to start over again.



For this game we're going to use images. You're free to use your own images, but I'm going to use these 3:



These images are made by [Kenney](#), who makes a lot of free assets that anyone can use in their games. Check him out!

Let's start with the 3 main callbacks, and load *classic*, the library we use to simulate classes.

```
function love.load()
```

```
    Object = require "classic"
```

```
end
```

```
function love.update(dt)
```

```
end
```

```
function love.draw()
```

```
end
```

Let's start with the player. Create a new file called `player.lua`.

We could make a base class for all our objects, but because it's such a short simple game we'll do without one. Though I encourage you to improve the code at the end of this chapter by adding a base class yourself.

Task: Create a moving player

Create a Player class:

```
--! file: player.lua
```

```
Player = Object:extend()
```

```
function Player:new()
```

```
end
```

I'm going to give the panda image to my player.

```
function Player:new()
```

```
    self.image = love.graphics.newImage("panda.png")
```

```
end
```

```
function Player:draw()
```

```
    love.graphics.draw(self.image)
```

```
end
```

Next let's make it possible to move our player with our arrow keys.

```
function Player:new()
```

```
    self.image = love.graphics.newImage("panda.png")
```

```
    self.x = 300
```

```
    self.y = 20
```

```
    self.speed = 500
```

```
    self.width = self.image:getWidth()
```

```
end
```

```
function Player:update(dt)
```

```
    if love.keyboard.isDown("left") then
```

```
        self.x = self.x - self.speed * dt
```

```
    elseif love.keyboard.isDown("right") then
```

```
        self.x = self.x + self.speed * dt
```

end

end

function Player:draw()

love.graphics.draw(self.image, self.x, self.y)

end

And now we should be able to move our player. Let's go back to main.lua and load our player.

--! file: main.lua

function love.load()

Object = require "classic"

require "player"

player = Player()

end

function love.update(dt)

player:update(dt)

end

function love.draw()

player:draw()

end

As you can see, we can move our player. But our player can move out of the window. Let's fix this with if-statements.

--! file: player.lua

```

function Player:update(dt)

    if love.keyboard.isDown("left") then

        self.x = self.x - self.speed * dt

    elseif love.keyboard.isDown("right") then

        self.x = self.x + self.speed * dt

    end

    --Get the width of the window

    local window_width = love.graphics.getWidth()

    --If the x is too far too the left then..

    if self.x < 0 then

        --Set x to 0

        self.x = 0

    --Else, if the x is too far to the right then..

    elseif self.x > window_width then

        --Set the x to the window's width.

        self.x = window_width

    end

end

```

Oops, our player can still move too far to the right. We need to include our width when checking if we're hitting the right wall.

```

--If the left side is too far too the left then..

if self.x < 0 then

    --Set x to 0

```

```
self.x = 0
```

```
--Else, if the right side is too far to the right then..
```

```
elseif self.x + self.width > window_width then
```

```
    --Set the right side to the window's width.
```

```
    self.x = window_width - self.width
```

```
end
```

And now it's fixed. Our player can't move out of the window anymore.

Task: Create a moving enemy

Now let's make the Enemy class. Create a new file called enemy.lua, and type the following:

```
--! file: enemy.lua
```

```
Enemy = Object:extend()
```

```
function Enemy:new()
```

```
end
```

I'm going to give the enemy the snake image, and make it move by itself.

```
function Enemy:new()
```

```
    self.image = love.graphics.newImage("snake.png")
```

```
    self.x = 325
```

```
    self.y = 450
```

```
    self.speed = 100
```

```
end
```

```
function Enemy:update(dt)

    self.x = self.x + self.speed * dt

end
```

```
function Enemy:draw()

    love.graphics.draw(self.image, self.x, self.y)

end
```

We need to make the enemy bounce against the walls, but let's load it first.

--! file: main.lua

```
function love.load()

    Object = require "classic"

    require "player"

    require "enemy"


    player = Player()

    enemy = Enemy()

end
```

```
function love.update(dt)

    player:update(dt)

    enemy:update(dt)

end
```

```
function love.draw()

    player:draw()

    enemy:draw()

end
```

end

Okay now we can see the enemy move, and we can see it move out of our window. Let's make sure it doesn't move out of our window like with Player.

function Enemy:new()

```
self.image = love.graphics.newImage("snake.png")
```

```
self.x = 325
```

```
self.y = 450
```

```
self.speed = 100
```

```
self.width = self.image:getWidth()
```

```
self.height = self.image:getHeight()
```

end

function Enemy:update(dt)

```
self.x = self.x + self.speed * dt
```

```
local window_width = love.graphics.getWidth()
```

```
if self.x < 0 then
```

```
    self.x = 0
```

```
elseif self.x + self.width > window_width then
```

```
    self.x = window_width - self.width
```

```
end
```

end

Our enemy stops at the wall, but we want to make it bounce. How are we going to make it do that? It hits the right wall, and then what? It should move to the other direction. How do we make it move to the other direction? By changing the value of speed. And what should the value of speed become? It shouldn't be 100 but -100.

So should we do `self.speed = -100`? Well no. Because like I said before we'll make the enemy speed up as it gets hit, and this way it would reset its speed when it bounces. Instead, we should invert the value of speed. So speed becomes `-speed`. In other words, if the speed were to be increased to 120, it would then become -120.

And what if it hits the left wall? At that point speed is a negative number, and we should turn it into a positive number. How can we do that? Well, [negative times negative makes positive](#). So if we say that speed, which at that point is a negative number, becomes `-speed`, it will turn into a positive number.

function Enemy:update(dt)

```
    self.x = self.x + self.speed * dt
```

```
    local window_width = love.graphics.getWidth()
```

```
    if self.x < 0 then
```

```
        self.x = 0
```

```
        self.speed = -self.speed
```

```
    elseif self.x + self.width > window_width then
```

```
        self.x = window_width - self.width
```

```
        self.speed = -self.speed
```

```
    end
```

```
end
```

Alright we got a player and a moving enemy, now all that's left is the bullet.

Task: Be able to shoot bullets

Create a new file called `bullet.lua`, and write the following code:

```
--! file: bullet.lua
```

```
Bullet = Object:extend()
```

```
function Bullet:new()  
    self.image = love.graphics.newImage("bullet.png")  
end
```

```
function Bullet:draw()  
    love.graphics.draw(self.image)  
end
```

The bullets should move vertical instead of horizontal.

--We pass the x and y of the player.

```
function Bullet:new(x, y)  
    self.image = love.graphics.newImage("bullet.png")  
    self.x = x  
    self.y = y  
    self.speed = 700  
  
    --We'll need these for collision checking  
    self.width = self.image:getWidth()  
    self.height = self.image:getHeight()  
  
end
```

```
function Bullet:update(dt)  
    self.y = self.y + self.speed * dt  
  
end
```

```
function Bullet:draw()  
    love.graphics.draw(self.image, self.x, self.y)
```

end

Now we need to be able to shoot bullets. In main.lua load the file and create a table.

--! file: main.lua

function love.load()

Object = require "classic"

require "player"

require "enemy"

require "bullet"

player = Player()

enemy = Enemy()

listOfBullets = {}

end

Now we give Player a function that creates a bullet when space is pressed.

--! file: player.lua

function Player:keyPressed(key)

--If the spacebar is pressed

if key == "space" **then**

--Put a new instance of Bullet inside listOfBullets.

table.insert(listOfBullets, Bullet(self.x, self.y))

end

end

And we need to call this function in the love.keypressed callback.

--! file: main.lua

function love.keypressed(key)

player:keyPressed(key)

end

And now we need to iterate through the table and update/draw all the bullets.

function love.load()

Object = require "classic"

require "player"

require "enemy"

require "bullet"

player = Player()

enemy = Enemy()

listOfBullets = {}

end

function love.update(dt)

player:update(dt)

enemy:update(dt)

for i,v in ipairs(listOfBullets) do

v:update(dt)

end

end

function love.draw()

player:draw()

enemy:draw()

```
for i,v in ipairs(listOfBullets) do  
    v:draw()  
  
end  
  
end
```

Awesome, our player can now shoot bullets.

Task: Make bullets affect the enemy's speed

Now we need to make it so that the snake can get hit by the bullet. We give Bullet a collision detection function.

--! file: bullet.lua

```
function Bullet:checkCollision(obj)
```

```
end
```

Do you still know how to do it? Do you still know the 4 conditions that need to be true to assure collision is happening?

Instead of returning true and false, we increase the enemy's speed. We give the property dead to the bullet, which we'll use to remove it from the list.

```
function Bullet:checkCollision(obj)
```

```
    local self_left = self.x
```

```
    local self_right = self.x + self.width
```

```
    local self_top = self.y
```

```
    local self_bottom = self.y + self.height
```

```
    local obj_left = obj.x
```

```
    local obj_right = obj.x + obj.width
```

```
    local obj_top = obj.y
```

```
    local obj_bottom = obj.y + obj.height
```

```
if self_right > obj_left
and self_left < obj_right
and self_bottom > obj_top
and self_top < obj_bottom then
    self.dead = true
```

```
    --Increase enemy speed
```

```
    obj.speed = obj.speed + 50
```

```
end
```

```
end
```

Now we need to call checkCollision in main.lua.

```
function love.update(dt)
```

```
    player:update(dt)
```

```
    enemy:update(dt)
```

```
for i,v in ipairs(listOfBullets) do
```

```
    v:update(dt)
```

```
    --Each bullets checks if there is collision with the enemy
```

```
    v:checkCollision(enemy)
```

```
end
```

```
end
```

And next we need to destroy the bullets that are dead.

```
function love.update(dt)
```

```
    player:update(dt)
```

```
enemy:update(dt)
```

```
for i,v in ipairs(listOfBullets) do
```

```
    v:update(dt)
```

```
    v:checkCollision(enemy)
```

```
    --If the bullet has the property dead and it's true then..
```

```
    if v.dead then
```

```
        --Remove it from the list
```

```
        table.remove(listOfBullets, i)
```

```
    end
```

```
end
```

```
end
```

Last thing to do is to restart the game when we miss the enemy. We need to check if the bullet is out of the screen.

```
--! file: bullet.lua
```

```
function Bullet:update(dt)
```

```
    self.y = self.y + self.speed * dt
```

```
    --If the bullet is out of the screen
```

```
    if self.y > love.graphics.getHeight() then
```

```
        --Restart the game
```

```
        love.load()
```

```
    end
```

```
end
```

And let's test it out. You might notice that when you hit the enemy while it's moving to the left, it slows down. This is because the enemy's speed is at that point a negative number.

So by increasing the number the enemy slows down. To fix this we need to check if the enemy's speed is a negative number or not.

function Bullet:checkCollision(obj)

local self_left = self.x

local self_right = self.x + self.width

local self_top = self.y

local self_bottom = self.y + self.height

local obj_left = obj.x

local obj_right = obj.x + obj.width

local obj_top = obj.y

local obj_bottom = obj.y + obj.height

if self_right > obj_left

and self_left < obj_right

and self_bottom > obj_top

and self_top < obj_bottom **then**

self.dead = **true**

--Increase enemy speed

if obj.speed > 0 **then**

obj.speed = obj.speed + 50

else

obj.speed = obj.speed - 50

end

end

end

And with that our game is finished. Or is it? You should try adding features to the game yourself. Or make a whole new game. It doesn't matter as long as you keep learning and keep making games!

Summary

A game is essentially a bunch of problems that need to be solved.